

Привет ИИ! Делаем спринт. Твоя роль — джуниор-разработчик, а я - твой лид. Я жду от тебя качественный код: чистая архитектура, грамотная обработка ошибок, отсутствие антипаттернов, JSDoc-комментариями к каждому методу, осмысленными именами переменных. Не задавай лишних уточняющих вопросов, — в ТЗ всё указано, поэтому спрашивай только если что-то серьезное. Для мелких вопросов сам принимай разумное решение и пиши комментарий // DECISION: ... с объяснением.

КОНТЕКСТ ПРОЕКТА

Ссылка на наш репозиторий, обязательно ознакомься с содержимым: <https://github.com/yakexerr/STD/tree/docs/SD-diagrams-and-db-scheme>

Проект: STD — веб-платформа для выпуска, учёта и проверки подлинности

дипломов и сертификатов на базе собственного блокчейн-реестра.

Репозиторий: <https://github.com/yakexerr/STD>

Стек: Node.js (backend + blockchain), React (frontend), MySQL (off-chain).

Суть: ВУЗ выпускает диплом → система считает SHA-256 хэш PDF-файла →

записывает хэш в блокчейн. Работодатель загружает PDF → система считает

хэш → ищет его в блокчейне → возвращает статус VALID / REVOKED / NOT_FOUND.

Блокчейн — это НЕ распределённая сеть. Это один JSON-файл (blockchain.json)

с массивом блоков, сцепленных хэшами. Три типа блоков: BEGIN, ISSUE, REVOKE.

Гибридная модель хранения:

- On-chain (blockchain.json): хэши документов, статусы, ID.
- Off-chain (MySQL): пользователи, метаданные документов,

история запросов.

СТРУКТУРА ПАПООК ПРОЕКТА (создавай файлы строго по этим путям)

STD/

```
├── blockchain/
│   ├── blockchain.js      ← Класс Block + Класс Blockchain
│   └── blockchain.json    ← Создаётся автоматически
├── package.json
├── server/
│   ├── server.js          ← Express entry point (порт 5000)
│   └── db.js              ← MySQL connection pool
├── (mysql2/promise)
│   ├── config.js          ← Конфигурация (порты, секреты,
│   │                       пути)
│   └── routes/
│       ├── auth.js        ← POST /api/auth/register,
│       │                   /api/auth/login
│       ├── documents.js    ← POST /api/documents/issue,
│       │                   /revoke; GET /my
│       └── validation.js   ← POST /api/validation/check;
│                           GET /history
│   ├── middleware/
│       └── auth.js         ← JWT-верификация, проверка
│                           роли
│   ├── services/
│       └── documentService.js ← Логика: multer → uuid →
│                           хэш → БД → блокчейн
│       └── validationService.js ← Логика: multer → хэш →
│                           блокчейн → БД
│   └── sql/
│       └── init.sql        ← CREATE DATABASE + CREATE
```

TABLES

```
| | └── uploads/           ← Папка для загруженных PDF (в
.gitignore)
| |   └── .gitkeep
| |   └── package.json
|
| └── frontend/           ← Уже инициализирован (React)
| | └── src/
| |   └── App.jsx         ← Роутинг
| |   └── pages/
| |       └── AuthPage.jsx ←>Login / регистрация
| |       └── AccountPage.jsx ← Личный кабинет
| |       └── ValidationPage.jsx ← Публичная проверка
документа
| |   └── services/
| |       └── api.js       ← axios instance, baseUrl
http://localhost:5000/api
|   └── package.json
|
└── README.md
```

РАЗБИВКА РАБОТЫ НА ЧАСТИ

Работа разделена на 4 части. В каждом сообщении я буду давать ТЗ для одной части. Ты выполняешь ТОЛЬКО текущую часть. Не пиши код для будущих частей, пока не получишь ТЗ.

Часть 1: blockchain/blockchain.js + blockchain/package.json

Часть 2: server/ (server.js, db.js, config.js, sql/init.sql, routes/, middleware/, services/, package.json)

Часть 3: frontend/ (App.jsx, страницы, api.js)

Часть 4: Тесты блокчейна + docker-compose.yml + README.md

ТЗ ЧАСТИ 1: ЯДРО БЛОКЧЕЙНА

Создай два файла:

- 1) blockchain/package.json
 - 2) blockchain/blockchain.js
-
-

1) blockchain/package.json

```
{  
  "name": "std-blockchain",  
  "version": "1.0.0",  
  "description": "Blockchain core for STD diploma verification  
platform",  
  "main": "blockchain.js",  
  "scripts": {  
    "test": "node -test"  
  },  
  "dependencies": {}  
}
```

Зависимостей НЕТ. Используем только встроенный модуль crypto и fs.

2) blockchain/blockchain.js

Экспортируй два класса: Block и Blockchain.

—— КЛАСС Block ——

Constructor принимает: index, timestamp, type, data, previousHash.

Поле hash вычисляется в конструкторе автоматически.

Метод calculateHash():

- Использует crypto.createHash('sha256').
- Строка для хэша формируется СТРОГО так:
String(this.index) + this.timestamp + this.type
+ deterministicStringify(this.data) + this.previousHash
- deterministicStringify — это JSON.stringify с СОРТИРОВКОЙ ключей

по алфавиту на всех уровнях вложенности. Реализуй как приватную

функцию внутри модуля (не экспортируй). Пример:

```
function deterministicStringify(obj) {  
  return JSON.stringify(obj, Object.keys(obj).sort());  
}
```

НО: Object.keys(obj).sort() работает только для верхнего уровня.

Для вложенных объектов нужна рекурсивная сортировка.

Реализуй

рекурсивную версию sortKeysDeep(obj), которая возвращает новый

объект с отсортированными ключами на всех уровнях, а затем

делай JSON.stringify(sortKeysDeep(obj)).

Структура блока BEGIN (Genesis):

index: 0, timestamp: ISO-строка, type: "BEGIN",
data: {}, previousHash: "0", hash: вычисленный.

Структура блока ISSUE:

index: N, timestamp: ISO, type: "ISSUE",
data: { documentId, documentHash, issuerId, holderId },

previousHash: хэш предыдущего блока, hash: вычисленный.

Структура блока REVOKE:

index: N, timestamp: ISO, type: "REVOKE",

data: { documentId },

previousHash: хэш предыдущего блока, hash: вычисленный.

—— КЛАСС Blockchain ——

Constructor принимает опциональный параметр filePath (по умолчанию: path.join(__dirname, 'blockchain.json')).

В конструкторе вызывает this.load().

Поля:

- this.chain — массив блоков (Block[])
- this.filePath — путь к JSON-файлу

Метод load():

- Если файл НЕ существует или пустой → вызывает createGenesisBlock(), затем save().
- Если файл существует → читает JSON, парсит, создаёт объекты Block из каждого элемента (пересчитывает hash в конструкторе Block).
Затем вызывает isValid(). Если isValid() === false → throw new Error('Blockchain integrity check failed on load').

Метод save():

- Записывает this.chain в this.filePath через fs.writeFileSync с JSON.stringify(this.chain, null, 2).

Метод createGenesisBlock():

- Создает Block(0, new Date().toISOString(), "BEGIN", {}, "0").
- this.chain = [genesisBlock].

Метод getLatestBlock():

- Возвращает this.chain[this.chain.length - 1].

Метод addBlock(type, data):

- Приватный метод (не экспортируй отдельно, но используй

внутри).

- Создает новый Block:
index = getLatestBlock().index + 1
timestamp = new Date().toISOString()
type = переданный type
data = переданный data
previousHash = getLatestBlock().hash
- Пушит в this.chain.
- Вызывает this.save().
- Возвращает созданный блок.

Метод issueDocument(documentId, documentHash, issuerId, holderId):

- Проверяет ВСЮ цепочку: ищет блоки ISSUE с совпадающим documentId
ИЛИ совпадающим documentHash.
- Если найден → throw new Error(`Document already exists: ...`).
- Вызывает addBlock("ISSUE", { documentId, documentHash, issuerId, holderId }).
- Возвращает созданный блок.

Метод revokeDocument(documentId):

- Ищет блок ISSUE с данным documentId.
- Если НЕ найден → throw new Error(`Document \${documentId} not found`).
- Ищет блок REVOKE с данным documentId.
- Если найден → throw new Error(`Document \${documentId} already revoked`).
- Вызывает addBlock("REVOKE", { documentId }).
- Возвращает созданный блок.

Метод checkDocument(documentHash):

- Сканирует цепочку, ищет блок ISSUE с data.documentHash === documentHash.
- Если НЕ найден → return { isValid: false, status: "NOT_FOUND" }.
- Если найден → берёт documentId из этого блока.
- Ищет блок REVOKE с data.documentId === documentId.
- Если REVOKE найден → return { isValid: false, status: "REVOKED" }.
- Иначе → return { isValid: true, status: "VALID" }.

Метод isValid():

- Проверяет genesis: index === 0, previousHash === "0", type === "BEGIN", пересчитанный hash === block.hash.
- Цикл от i = 1 до chain.length - 1:
 - chain[i].previousHash === chain[i-1].hash
 - chain[i].index === chain[i-1].index + 1
 - chain[i].calculateHash() === chain[i].hash
- Если всё ок → return true. Иначе → return false.

Метод getChainLength():

- return this.chain.length.

—— ЭКСПОРТ ——

```
module.exports = { Block, Blockchain };
```

—— ТРЕБОВАНИЯ К КАЧЕСТВУ ——

- JSDoc-комментарий (@param, @returns, @throws) к КАЖДОМУ публичному методу.
- Обработка ошибок: все throw new Error с осмысленными сообщениями.
- Никаких console.log в продакшн-коде. Только throw или return.
- Код должен быть полностью рабочим. Никаких TODO, заглушек, "...".
- Используй 'use strict' в начале файла.
- require('crypto'), require('fs'), require('path') в начале файла.

Выполни Часть 1. Жду от тебя полный код обоих файлов.